

Sentiment Analysis Tool Project

This project is a simple sentiment analysis application that classifies text into:

- Positive
- Negative
- Neutral

It works for:

- Social media posts
- Product reviews
- Restaurant reviews

Project Structure

```
sentiment-analysis-tool/  
|  
├─ app.py  
├─ train_model.py  
├─ sentiment_model.pkl  
├─ vectorizer.pkl  
├─ requirements.txt  
├─ README.md  
└─ templates/  
    └─ index.html
```

File: requirements.txt

```
flask  
scikit-learn  
pandas  
numpy  
joblib
```

File: train_model.py

```
import pandas as pd  
from sklearn.feature_extraction.text import TfidfVectorizer
```

```

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score
import joblib

# Sample dataset

data = {
    'text': [
        'I love this product',
        'Amazing quality and service',
        'The food was delicious',
        'Very happy with the purchase',
        'This is terrible',
        'Worst experience ever',
        'The food was cold and tasteless',
        'I hate this item',
        'It is okay',
        'Average experience',
        'Nothing special',
        'The product is fine'
    ],
    'sentiment': [
        'Positive',
        'Positive',
        'Positive',
        'Positive',
        'Negative',
        'Negative',
        'Negative',
        'Negative',
        'Neutral',
        'Neutral',
        'Neutral',
        'Neutral'
    ]
}

# Create dataframe

df = pd.DataFrame(data)

# Features and labels
X = df['text']
y = df['sentiment']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

# Vectorizer
vectorizer = TfidfVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Model
model = MultinomialNB()
model.fit(X_train_vec, y_train)

# Prediction
predictions = model.predict(X_test_vec)

# Accuracy
accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy * 100:.2f}%')

# Save model and vectorizer
joblib.dump(model, 'sentiment_model.pkl')
joblib.dump(vectorizer, 'vectorizer.pkl')

print('Model and vectorizer saved successfully!')

```

File: app.py

```

from flask import Flask, render_template, request
import joblib

# Load trained model and vectorizer
model = joblib.load('sentiment_model.pkl')
vectorizer = joblib.load('vectorizer.pkl')

app = Flask(__name__)

# Prediction function
def predict_sentiment(text):
    text_vector = vectorizer.transform([text])
    prediction = model.predict(text_vector)
    return prediction[0]

@app.route('/', methods=['GET', 'POST'])
def home():
    sentiment = None

    if request.method == 'POST':
        user_text = request.form['text']
        sentiment = predict_sentiment(user_text)

```

```
    return render_template('index.html', sentiment=sentiment)

if __name__ == '__main__':
    app.run(debug=True)
```

File: templates/index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Sentiment Analysis Tool</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f5f5f5;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }

        .container {
            background: white;
            padding: 30px;
            border-radius: 10px;
            width: 500px;
            box-shadow: 0 0 10px rgba(0,0,0,0.2);
        }

        textarea {
            width: 100%;
            height: 120px;
            padding: 10px;
            font-size: 16px;
        }

        button {
            margin-top: 15px;
            padding: 10px 20px;
            font-size: 16px;
            background: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
    </style>

```

```

        h2 {
            text-align: center;
        }

        .result {
            margin-top: 20px;
            font-size: 20px;
            text-align: center;
            font-weight: bold;
        }
    </style>
</head>
<body>

<div class="container">
    <h2>Sentiment Analysis Tool</h2>

    <form method="POST">
        <textarea name="text" placeholder="Enter review or social post..."></
textarea>
        <button type="submit">Analyze Sentiment</button>
    </form>

    {% if sentiment %}
        <div class="result">
            Sentiment: {{ sentiment }}
        </div>
    {% endif %}
</div>

</body>
</html>

```

File: README.md

```
# Sentiment Analysis Tool
```

This project is a machine learning based sentiment analysis tool using Python and Flask.

```
## Features
```

- Classifies text as Positive, Negative, or Neutral
- Simple web interface
- Uses TF-IDF and Naive Bayes
- Suitable for:
 - Social media posts
 - Product reviews

```
- Restaurant reviews

## Installation

### Step 1: Install dependencies

```bash

pip install -r requirements.txt
```

### Step 2: Train the model

```
python train_model.py
```

### Step 3: Run the application

```
python app.py
```

### Step 4: Open browser

```
http://127.0.0.1:5000/
```

## Example Inputs

- "The food was amazing!" → Positive
- "Worst product ever." → Negative
- "It is okay." → Neutral ``

---

## How the Project Works

1. User enters text into the web page.
2. TF-IDF converts text into numerical features.
3. Naive Bayes model predicts sentiment.
4. Result is displayed instantly.

---

## Future Improvements

- Add deep learning models (LSTM/BERT)
- Support multiple languages
- Add sentiment score percentage
- Connect with Twitter or Instagram APIs
- Store reviews in a database